

Practical No. 08

Title: Designing Test Suites for Pharmacy Management System

Aim:

To design and implement test cases and test suites for verifying the functionality of a Pharmacy Management System and locating possible bugs using different software testing techniques.

Objective:

After completing this experiment, students will be able to:

- Understand different techniques of software testing.
 - Design unit test cases for verifying the functionality of a Pharmacy Management System.
 - Identify and document software bugs and inconsistencies through systematic testing.
-

Introduction:

A Pharmacy Management System is a software application used to automate pharmacy operations like medicine inventory, sales, billing, supplier management, and customer records. To ensure the smooth functioning of such a system, software testing is crucial. It helps in verifying system functionalities, identifying defects, and ensuring that the application meets both functional and user requirements. This practical focuses on designing and implementing test cases and test suites for various modules of a Pharmacy Management System.

Theory:

Software Testing:

Software testing is the process of executing a program to identify any errors, gaps, or missing requirements. It verifies that the software works as expected and delivers reliable results.

Verification and Validation:

- **Verification:** Confirms whether the product is built correctly according to the design specifications.
- **Validation:** Checks whether the software meets the business and customer requirements.

Standards for Software Test Documentation:

Standardized documentation like test plans, test cases, test suites, and test reports ensures a systematic, consistent, and reliable testing process.

Testing Frameworks:

Frameworks like Selenium, JUnit, and PyTest are commonly used to automate the execution of test cases and record results.

Need for Software Testing:

- Detect bugs early in development.
- Ensure software reliability, security, and performance.
- Confirm that the software meets client and end-user expectations.

Test Cases and Test Suites:

- **Test Case:** A set of input values, execution conditions, and expected results for a particular software feature.
- **Test Suite:** A collection of test cases grouped to test software systematically.

Types of Software Testing:

- **Unit Testing:** Tests individual modules like billing, stock management, etc.
- **Integration Testing:** Tests interaction between modules such as medicine sales and inventory updates.
- **System Testing:** Tests the complete Pharmacy Management System as a whole.

Case Study:

Case Study Title: *Pharmacy Management System — Medicine Sale Module*

Description:

The Medicine Sale module of a Pharmacy Management System allows the pharmacist to sell medicines to customers, verify stock availability, and generate bills. It should ensure valid inputs and handle errors gracefully.

Test Suite Name: *Medicine Sale Test Suite*

Test Cases:

Test Case ID	Description	Input	Expected Output	Actual Output	Status
TC_01	Sell medicine with valid medicine ID and quantity	Medicine ID: M101, Quantity: 5	Successful sale, bill generated, stock reduced	-	-
TC_02	Sell medicine with invalid medicine ID	Medicine ID: M999, Quantity: 2	"Medicine not found" error message	-	-
TC_03	Sell medicine with quantity exceeding stock	Medicine ID: M101, Quantity: 50	"Insufficient stock" error message	-	-

TC_04	Sell medicine with zero quantity	Medicine ID: M101, Quantity: 0	"Invalid quantity" error message	-	-
TC_05	Generate bill after successful sale	Medicine ID: M101, Quantity: 2	Bill generated with correct total amount	-	-

Execution:

Each test case is executed manually or via automation, comparing actual outcomes against expected results. Status is marked as *Pass* or *Fail* and recorded in the test report.

Conclusion:

In this practical, we explored the importance of software testing in maintaining the quality of a Pharmacy Management System. By designing structured test cases and test suites, we verified the proper functioning of the Medicine Sale module and identified potential issues such as invalid inputs and stock inconsistencies. This systematic approach to testing improves software reliability and ensures customer satisfaction.